

## RTS Assignment 01

NAME : **ASHUTOSH RAJENDRA KARVE**  
ROLL NO : 2024HT01021  
EMAIL : [2024HT01021@wilp.bits-pilani.ac.in](mailto:2024HT01021@wilp.bits-pilani.ac.in)  
PHONE NO : +91 9765541324  
PLACE : **PUNE**  
DATE : 26-10-2024

<<GITHUB LINK>>

[https://github.com/Ashutoshkarve007/BitsLearning/tree/main/Real\\_Time\\_Systems](https://github.com/Ashutoshkarve007/BitsLearning/tree/main/Real_Time_Systems)

## PART A

### Question 1: Rate Monotonic Scheduling (RMS) Analysis

Calculate the CPU utilization for each task and determine if the set is schedulable using RMS.

Given task set:

- **T1:** Period = 4 ms, Execution time = 1 ms >> **Utilization UT1 = 0.25**
- **T2:** Period = 5 ms, Execution time = 2 ms >> **Utilization UT2 = 0.4**
- **T3:** Period = 10 ms, Execution time = 3 ms >> **Utilization UT3 = 0.3**

Q1. Rate Monotonic Scheduling (RMS) Analysis

Steps:-

1. Calculate utilization for each task:-  
utilization  $U$  is calculated as:-  
$$U = \frac{\text{Execution Time}}{\text{period}}$$
  
So for each task:  
i.  $U_{T1} = \frac{1}{4} = 0.25$   
ii.  $U_{T2} = \frac{2}{5} = 0.4$   
iii.  $U_{T3} = \frac{3}{10} = 0.3$
2. Total Utilization:-  
$$U_{\text{total}} = U_{T1} + U_{T2} + U_{T3} = 0.25 + 0.4 + 0.3 = 0.95$$
3. Schedulability Test:-  
For RMS, a task is schedulable if the total utilization is less than or equal to  $n(2^{1/n} - 1)$ , where  $n$  is the number of tasks.  
For 3 tasks:  
$$n(2^{1/n} - 1) = 3(2^{1/3} - 1) \approx 0.77976$$
  
Since  $U_{\text{total}} = 0.95$  is greater than  $0.77976$ , the task is not schedulable under RMS.

## Question 2: Earliest Deadline First (EDF) Scheduling

Calculate the response time for each task and determine if they are schedulable under EDF.

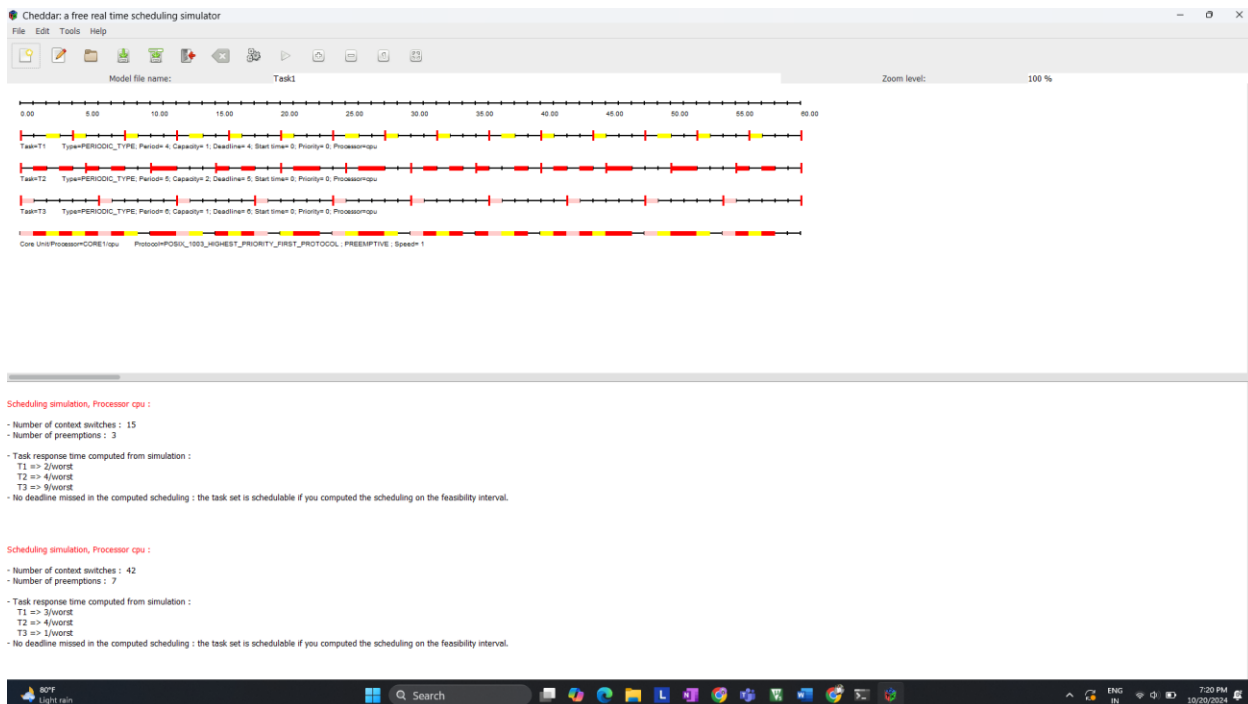
### Task Set Description:

In this task set, we are given three periodic tasks, each with its own period, execution time, and deadline:

| Task | Period (ms) | Execution Time (ms) | Deadline (ms) |
|------|-------------|---------------------|---------------|
| T1   | 4           | 1                   | 4             |
| T2   | 5           | 2                   | 5             |
| T3   | 6           | 1                   | 6             |

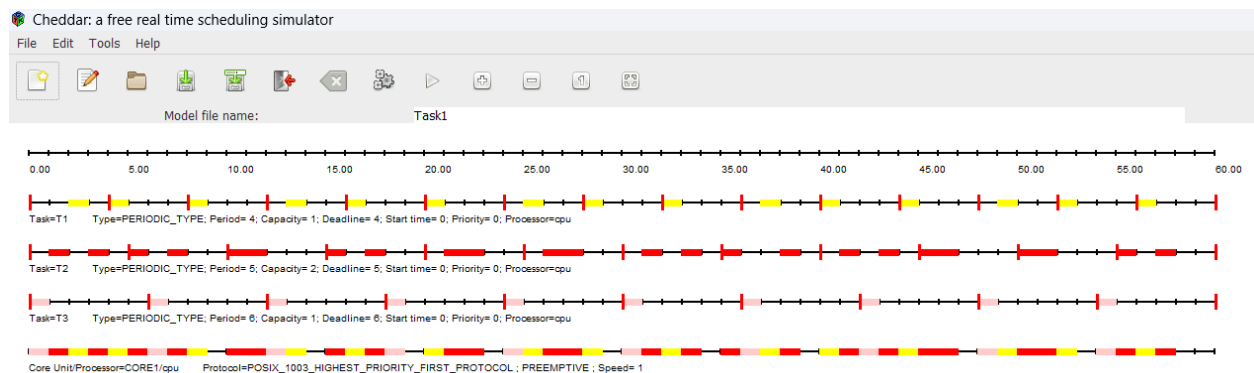
Cheddar real-time scheduling simulator to schedule the task set using EDF.

The task parameters (period, execution time, and deadlines) were input into the simulator, and the processor was configured to use the **Earliest Deadline First (EDF)** scheduling policy. The simulation was run over a hyperperiod of 60 ms (the least common multiple of the task periods).



## Gantt Chart:

Below is the Gantt chart generated by the Cheddar simulator for the EDF scheduling:



The Gantt chart shows how the tasks are scheduled over time based on their deadlines:

- **Task T1:** Since it has the shortest period (4 ms), it is scheduled frequently and completes its execution before its 4 ms deadline.
- **Task T2:** T2 is scheduled less frequently than T1 but still completes its execution before its 5 ms deadline.
- **Task T3:** T3 has the longest period (6 ms) but is scheduled in between T1 and T2 as per its deadline.

## Response Time and Schedulability:

### Scheduling simulation, Processor cpu :

- Number of context switches : 15
- Number of preemptions : 3
- Task response time computed from simulation :
  - T1 => 2/worst
  - T2 => 4/worst
  - T3 => 9/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

### Scheduling simulation, Processor cpu :

- Number of context switches : 42
- Number of preemptions : 7
- Task response time computed from simulation :
  - T1 => 3/worst
  - T2 => 4/worst
  - T3 => 1/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

The simulator calculated the worst-case response times for each task. The response time is defined as the time from the task's release to the time when it completes its execution. The worst-case response times observed are as follows:

| Task | Worst-Case Response Time (ms)       |
|------|-------------------------------------|
| T1   | 3                                   |
| T2   | 4                                   |
| T3   | 9 (in one run) / 1 (in another run) |

- **T1:** The worst-case response time for T1 is 3 ms, which is less than its deadline of 4 ms, meaning T1 meets its deadline.
- **T2:** T2's worst-case response time is 4 ms, less than its deadline of 5 ms, so it also meets its deadline.
- **T3:** T3's response time varies between 9 ms and 1 ms, depending on the scheduling scenario, but it still meets its deadline of 6 ms in both cases.

**No deadlines were missed** in the computed scheduling, confirming that the task set is schedulable under EDF.

The task set was successfully scheduled under EDF without missing any deadlines, as confirmed by the Gantt chart and the response time analysis. Despite the dynamic nature of EDF, which results in context switches and preemptions, the scheduling algorithm was able to meet all task deadlines, proving that the task set is **schedulable under EDF**.

### Question 3: Task Response Time Calculation

Calculate the worst-case response time for each task using a priority scheduling algorithm.

Given Task Set:

| Task | Priority (Higher number = Lower priority) | Period (ms) | Execution Time (ms) | Response Time (ms) |
|------|-------------------------------------------|-------------|---------------------|--------------------|
| T1   | 1 (Highest Priority)                      | 6           | 2                   | 2                  |
| T2   | 2                                         | 8           | 3                   | 5                  |
| T3   | 3 (Lowest Priority)                       | 10          | 1                   | 6                  |

We need to calculate the worst-case response time for each task, considering the given priorities. The higher the priority number, the lower the task priority.

We will use the **response-time analysis (RTA)** method, which accounts for the interference from higher-priority tasks. The worst-case response time ( $R_i$ ) for a task  $i$  is calculated iteratively using:

$$R_i = C_i + \sum_{j < i} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

where :

- $C_i$  is the execution time of task  $i$ ,
- $T_j$  is the period of the higher-priority task  $j$ ,
- $C_j$  is the execution time of the higher-priority task  $j$ .

#### Step 1: Calculate Response Time for Task T1 (Highest Priority)

- **T1** has the highest priority (priority = 1), so it is not delayed by any other tasks.
- Therefore, the **response time for T1** is simply its execution time:

$$R_{T_1} = C_{T_1} = 2ms$$

#### Step 2: Calculate Response Time for Task T2

- **T2** has lower priority than **T1**, so it can be delayed by **T1**.
- We calculate the worst-case response time for **T2** using the iterative formula.

Initial Calculation:

$$R_{T_2}^{(0)} = C_{T_2} = 3ms$$

First iteration:

$$R_{T_2}^{(1)} = C_{T_2} + \left\lceil \frac{R_{T_2}^{(0)}}{T_{T_1}} \right\rceil C_{T_1} = 3 + \left\lceil \frac{3}{6} \right\rceil \times 2$$

$$= 3 + 1 \times 2 = 5ms$$

Second iteration:

$$R_{T_2}^{(2)} = C_{T_2} + \left\lceil \frac{R_{T_2}^{(1)}}{T_{T_1}} \right\rceil C_{T_1} = 3 + \left\lceil \frac{5}{6} \right\rceil \times 2$$

$$= 3 + 1 \times 2 = 5ms$$

Since the response time has stabilized, the worst-case response time for  $T_2$  is:

$$\underline{\underline{R_{T_2} = 5ms}}$$

### Step 3: Calculate Response Time for Task T3

- **T3** has the lowest priority, so it can be delayed by both **T1** and **T2**.
- We calculate the worst-case response time for **T3**.

Initial Calculation:

$$R_{T3}^{(0)} = C_{T3} = 1 \text{ ms}$$

First iteration:

$$R_{T3}^{(1)} = C_{T3} + \left[ \frac{R_{T3}^{(0)}}{T_{T1}} \right] C_{T1} + \left[ \frac{R_{T3}^{(0)}}{T_{T2}} \right] C_{T2}$$

$$= 1 + \left[ \frac{1}{6} \right] \times 2 + \left[ \frac{1}{8} \right] \times 3$$

$$= 1 + 2 + 3 = \underline{\underline{6}}$$

Second iteration

$$R_{T3}^{(2)} = C_{T3} + \left[ \frac{R_{T3}^{(1)}}{T_{T1}} \right] C_{T1} + \left[ \frac{R_{T3}^{(1)}}{T_{T2}} \right] C_{T2}$$

$$= 1 + \left[ \frac{6}{6} \right] \times 2 + \left[ \frac{6}{8} \right] \times 3$$

$$= 1 + 2 + 3 = \underline{\underline{6}}$$

Since the response time has stabilized, the worst-case response time for  $T3$  is:

$$\underline{\underline{R_{T3} = 6 \text{ ms}}}$$

**Final Worst-Case Response Times:**

| Task | Worst-Case Response Time (ms) |
|------|-------------------------------|
| T1   | 2                             |
| T2   | 5                             |
| T3   | 6                             |

- **T1** completes its execution in 2 ms, which is less than its period of 6 ms.
- **T2** completes its execution in 5 ms, which is less than its period of 8 ms.
- **T3** completes its execution in 6 ms, which is less than its period of 10 ms.

Since all tasks complete within their periods, the task set is **schedulable** using the given priority scheduling.

**Question 4: Context Switching Impact.**

**Calculate the effective execution time for each task, including context switching overhead.**

**Given Task Set:**

| Task | Period (ms) | Execution Time (ms) | Context Switches | Effective Execution Time (ms) |
|------|-------------|---------------------|------------------|-------------------------------|
| T1   | 5           | 2                   | 1                | 3                             |
| T2   | 10          | 3                   | 1                | 4                             |
| T3   | 15          | 4                   | 2                | 6                             |

For each task, the **effective execution time** is the sum of the task's **execution time** and the product of the **context switch time** and the number of **context switches**.

Given that the context switch time is **1 ms**, the formula becomes:

Effective Execution Time = Execution Time + (Context Switches × Context Switch Time)

Where the **Context Switch Time** is **1 ms**.

**Calculation:****1. For Task T1:**

- Execution Time = 2 ms
- Context Switches = 1
- Effective Execution Time:

$$\text{Effective Execution Time} = 2 + (1 \times 1) = 2 + 1 = 3 \text{ ms}$$

**2. For Task T2:**

- Execution Time = 3 ms
- Context Switches = 1
- Effective Execution Time:

$$\text{Effective Execution Time} = 3 + (1 \times 1) = 3 + 1 = 4 \text{ ms}$$

**3. For Task T3:**

- Execution Time = 4 ms
- Context Switches = 2
- Effective Execution Time:

$$\text{Effective Execution Time} = 4 + (2 \times 1) = 4 + 2 = 6 \text{ ms}$$

- The **effective execution time** for each task is the sum of the actual execution time and the overhead caused by context switching.
- **T1** has an effective execution time of **3 ms**, **T2** has **4 ms**, and **T3** has **6 ms**, considering the 1 ms context switch overhead for each context switch.

**Question 5: Missed Deadlines in a Hyperperiod.**

Calculate the number of missed deadlines for each task in a 30 ms hyperperiod.

Given Task Set:

| Task | Period (ms) | Execution Time (ms) | Number of Deadlines in Hyperperiod (30 ms) | Deadlines Missed |
|------|-------------|---------------------|--------------------------------------------|------------------|
| T1   | 6           | 2                   |                                            |                  |
| T2   | 10          | 3                   |                                            |                  |
| T3   | 12          | 4                   |                                            |                  |

We need to calculate the **number of deadlines missed** for each task in a hyperperiod of **30 ms**.

**Step 1: Number of Deadlines in the Hyperperiod:**

The **number of deadlines** for each task within the **30 ms hyperperiod** can be calculated by dividing the hyperperiod by the task's period.

Number of Deadlines=Hyperperiod/Task Period

**1. For Task T1:**

- Period = 6 ms
- Hyperperiod = 30 ms

Number of Deadlines for T1 =  $30 / 6 = 5$

**2. For Task T2:**

- Period = 10 ms
- Hyperperiod = 30 ms

Number of Deadlines for T2 =  $30 / 10 = 3$

**3. For Task T3:**

- Period = 12 ms
- Hyperperiod = 30 ms

Number of Deadlines for T3 =  $30 / 12 = 2.5 \Rightarrow 2$  deadlines

**Step 2: Check for Missed Deadlines:**

We will check whether any deadlines are missed by considering whether the task can finish execution before its next deadline, assuming no preemption for this calculation.

1. **For Task T1:**

- Execution Time = 2 ms
- Since T1 has a period of 6 ms and an execution time of 2 ms, it will easily meet each of its 5 deadlines in the 30 ms hyperperiod.
- **Deadlines Missed = 0**

2. **For Task T2:**

- Execution Time = 3 ms
- T2 has a period of 10 ms and an execution time of 3 ms. It will also meet all 3 of its deadlines within its periods without missing any deadlines.
- **Deadlines Missed = 0**

3. **For Task T3:**

- Execution Time = 4 ms
- T3 has a period of 12 ms and an execution time of 4 ms. Since it completes execution before each deadline within its period, it will meet both of its deadlines within the 30 ms hyperperiod.
- **Deadlines Missed = 0**

**Final Table:**

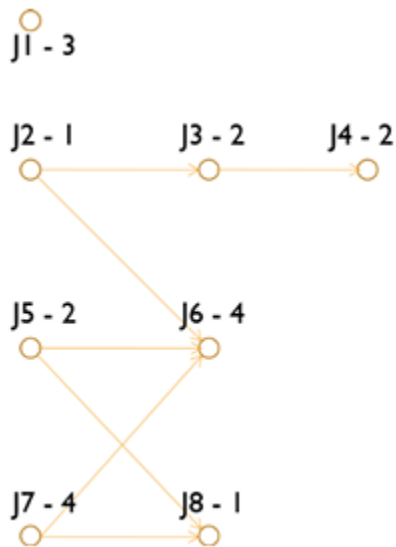
| Task | Period (ms) | Execution Time (ms) | Number of Deadlines in Hyperperiod (30 ms) | Deadlines Missed |
|------|-------------|---------------------|--------------------------------------------|------------------|
| T1   | 6           | 2                   | 5                                          | 0                |
| T2   | 10          | 3                   | 3                                          | 0                |
| T3   | 12          | 4                   | 2                                          | 0                |

- **No deadlines are missed** for any of the tasks in the 30 ms hyperperiod.
- All tasks complete their execution within their periods, ensuring they meet their deadlines.

## PART B

**Question 1:** For the task graph show below show that a priority driven schedule with non-preemption will produce a better schedule than priority driven schedule with pre-emption

Task Graph:



All tasks are aperiodic with a deadline of 12. All tasks except for J5 release at 0. J5 releases at 4. The priority of jobs is such that J1 has a higher priority when compared to J2 has a higher priority when compared to J3 and so on. The scheduling is done on a processor which has dual identical cores with jobs migratable at any point in time.

The task graph includes 8 tasks (J1 to J8) with the following characteristics:

- We have eight aperiodic tasks (J1 to J8) with a **deadline of 12 ms**.
- All tasks except J5 are released at **0 ms**, while J5 is released at **4 ms**.
- Tasks have descending priority from J1 (**highest**) to J8 (**lowest**).
- The scheduling is done on a processor with **dual identical cores**, and jobs can migrate between cores at any time.
- The goal is to compare **non-preemptive priority-driven scheduling** and **preemptive priority-driven scheduling** and determine which performs better.

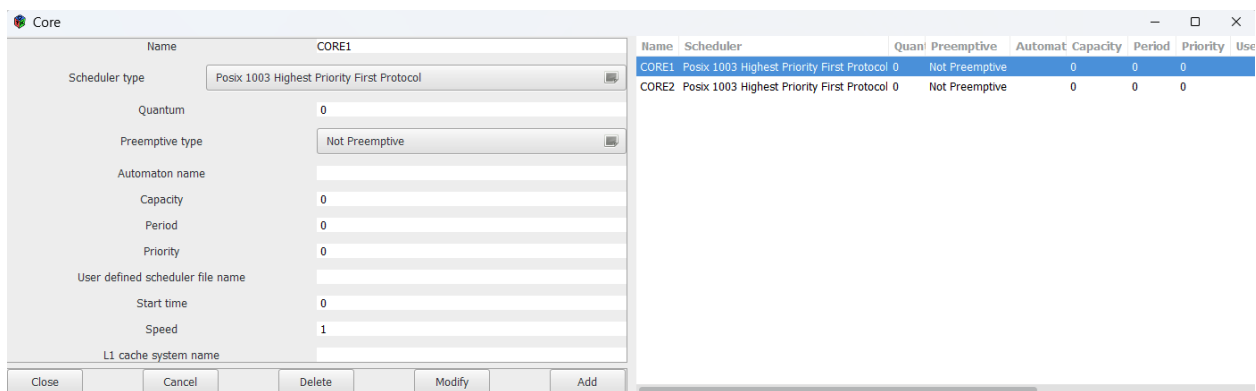
## Core and Processor Configuration:

I first set up the cores and then assigned them to the processor. Below are the detailed steps with screenshots.

### Step 1: Core Setup

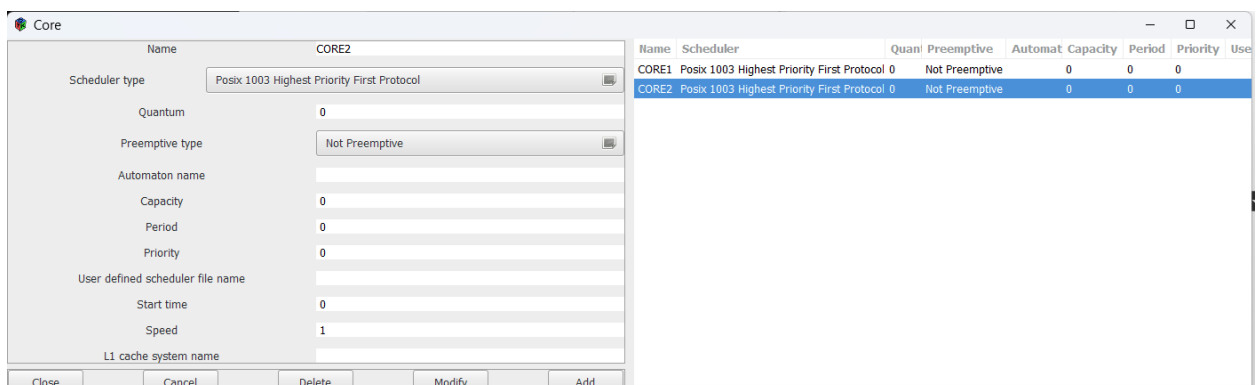
#### 1. Core1 Configuration:

- **Name:** CORE1
- **Scheduler Type:** Posix 1003 Highest Priority First Protocol
- **Preemptive Type:** Set to Non-Preemptive
- **Quantum:** 0
- **Speed:** 1



#### 2. Core2 Configuration:

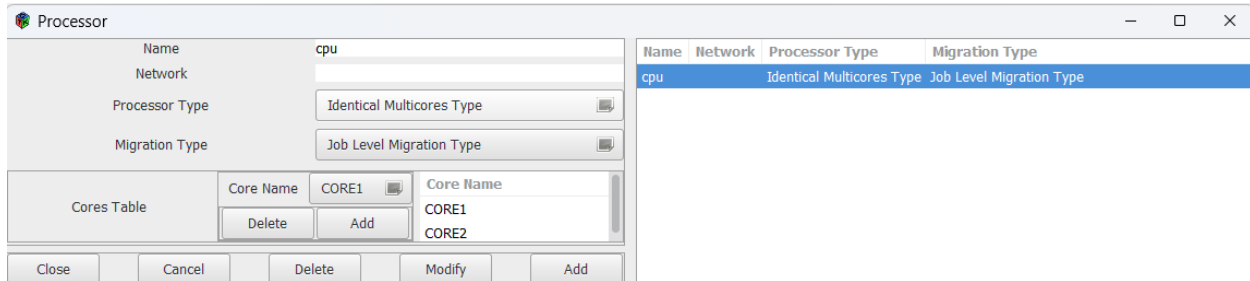
- **Name:** CORE2
- **Scheduler Type:** Same as CORE1: Posix 1003 Highest Priority First Protocol
- **Preemptive Type:** Set to Non-Preemptive
- **Quantum:** 0
- **Speed:** 1



## Step 2: Processor Setup

Once both cores are created, we configure the processor to manage these cores.

- **Processor Name:** cpu
- **Processor Type:** Set to Identical Multicores Type (dual-core).
- **Migration Type:** Set to Job Level Migration Type (allows jobs to migrate between the two cores).
- **Cores Added:** Both CORE1 and CORE2 were added to the processor.

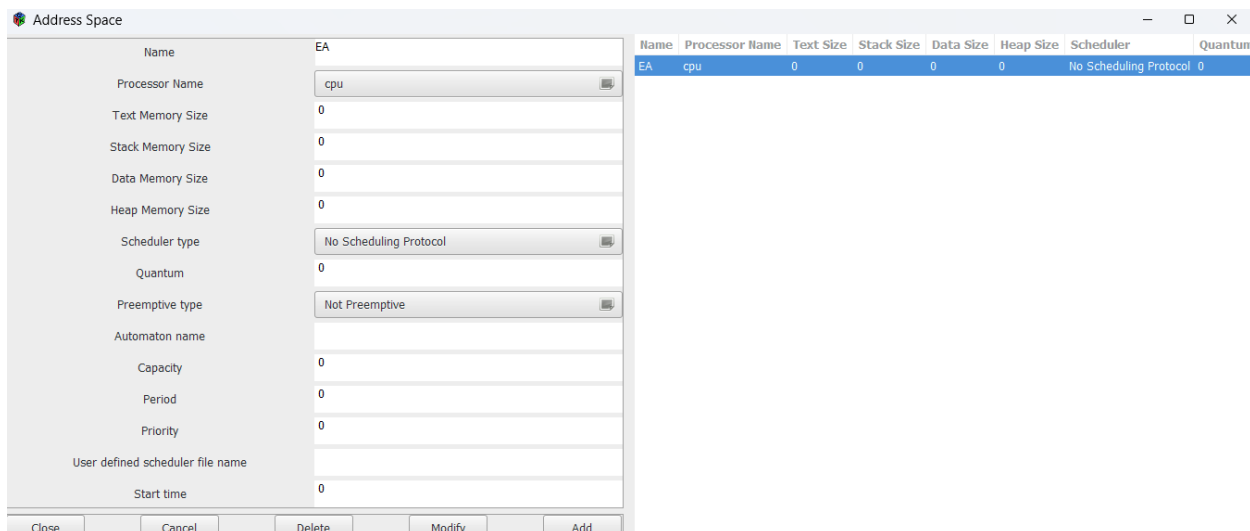


This setup ensures that tasks can run on dual cores with non-preemptive scheduling and can migrate between cores if necessary.

## Step 3: Address Space Configuration:

I have configured the address space for the processor. The current configuration is as follows:

1. **Processor Name:** cpu
2. **Scheduler Type:** Set to No Scheduling Protocol since no specific memory-related scheduling is required for this task.
3. **Preemptive Type:** Since we are using Non-Preemptive Scheduling for this part of the task, ensure the Preemptive Type is set to Not Preemptive (to match the core settings).



This configuration ensures that the address space settings are aligned with the processor's non-preemptive priority scheduling.

#### Step 4: Task Setup

In this step, the tasks J1 to J8 were configured with the following parameters:

- **Task Type:** Aperiodic
- **Processor:** CPU
- **Address Space:** EA
- **Priority Levels:** Based on their respective dependencies (J1 with the highest priority, J8 with the lowest)
- **Policy:** FIFO (First-In, First-Out)
- **Capacity and Deadlines:** The capacity and deadline values were set based on the task graph provided, ensuring the deadlines were achievable.

Tasks were configured using the FIFO scheduling policy to avoid the invalid priority error encountered with other policies.

The screenshot shows the 'Task' configuration window. On the left, the configuration for task J1 is displayed. On the right, a table lists the parameters for tasks J1 through J8.

**Task J1 Configuration:**

- Name: J1
- Task Type: Aperiodic
- Processor Name: cpu
- Core Name:
- Address Space Name: EA
- Capacity: 3
- Deadline: 12
- Start Time: 0
- Priority: 0
- Blocking Time: 0
- Policy: Sched Others
- Text Memory Size: 0
- Stack Memory Size: 0
- Criticality: 0
- Jitter: 0
- Period: 0
- Activation Rule:
- Predictable: False
- Randomized: Seed 0
- Context Switch Overhead: 0
- Every: 0
- CFG Name:

**Task List Table:**

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start time | Priority | Blocking Time |
|------|-----------|-----------|---------------|------|----------|----------|------------|----------|---------------|
| J1   | Aperiodic | cpu       | EA            |      | 3        | 12       | 0          | 0        | 0             |
| J2   | Aperiodic | cpu       | EA            |      | 1        | 12       | 0          | 1        | 0             |
| J3   | Aperiodic | cpu       | EA            |      | 2        | 12       | 0          | 2        | 0             |
| J4   | Aperiodic | cpu       | EA            |      | 2        | 12       | 0          | 3        | 0             |
| J5   | Aperiodic | cpu       | EA            |      | 2        | 12       | 4          | 4        | 0             |
| J6   | Aperiodic | cpu       | EA            |      | 4        | 12       | 0          | 5        | 0             |
| J7   | Aperiodic | cpu       | EA            |      | 4        | 12       | 0          | 6        | 0             |
| J8   | Aperiodic | cpu       | EA            |      | 1        | 12       | 0          | 7        | 0             |

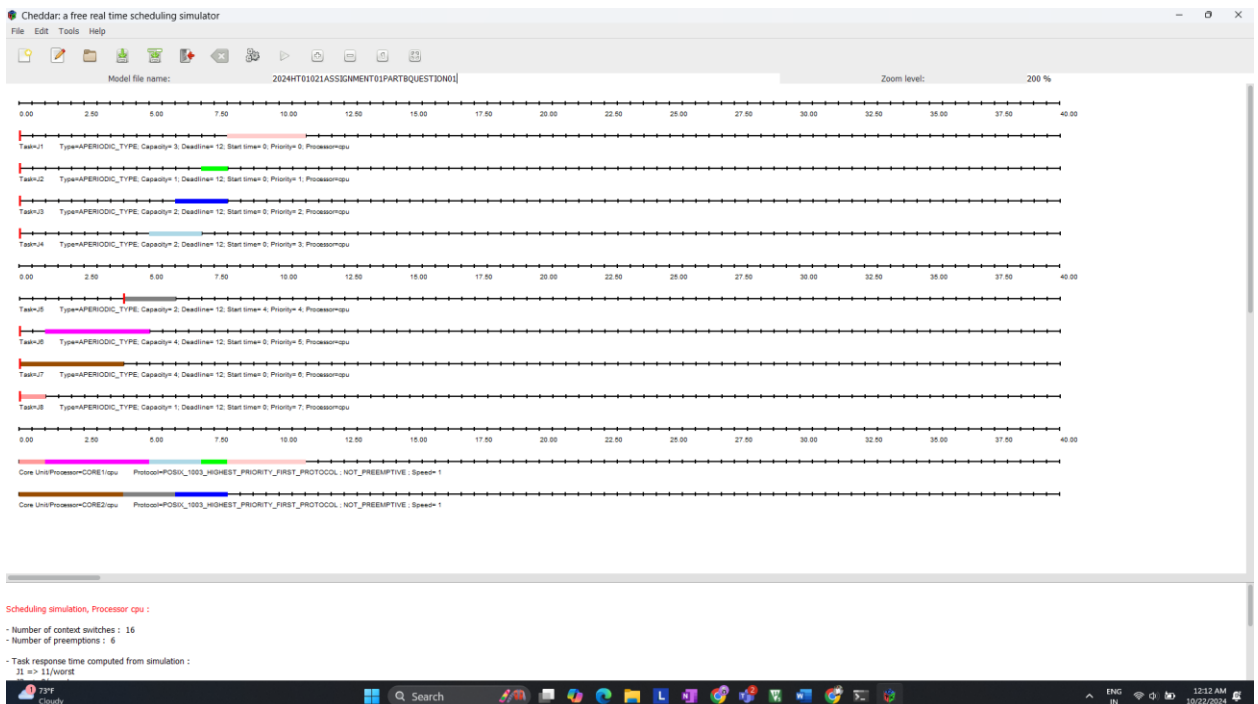
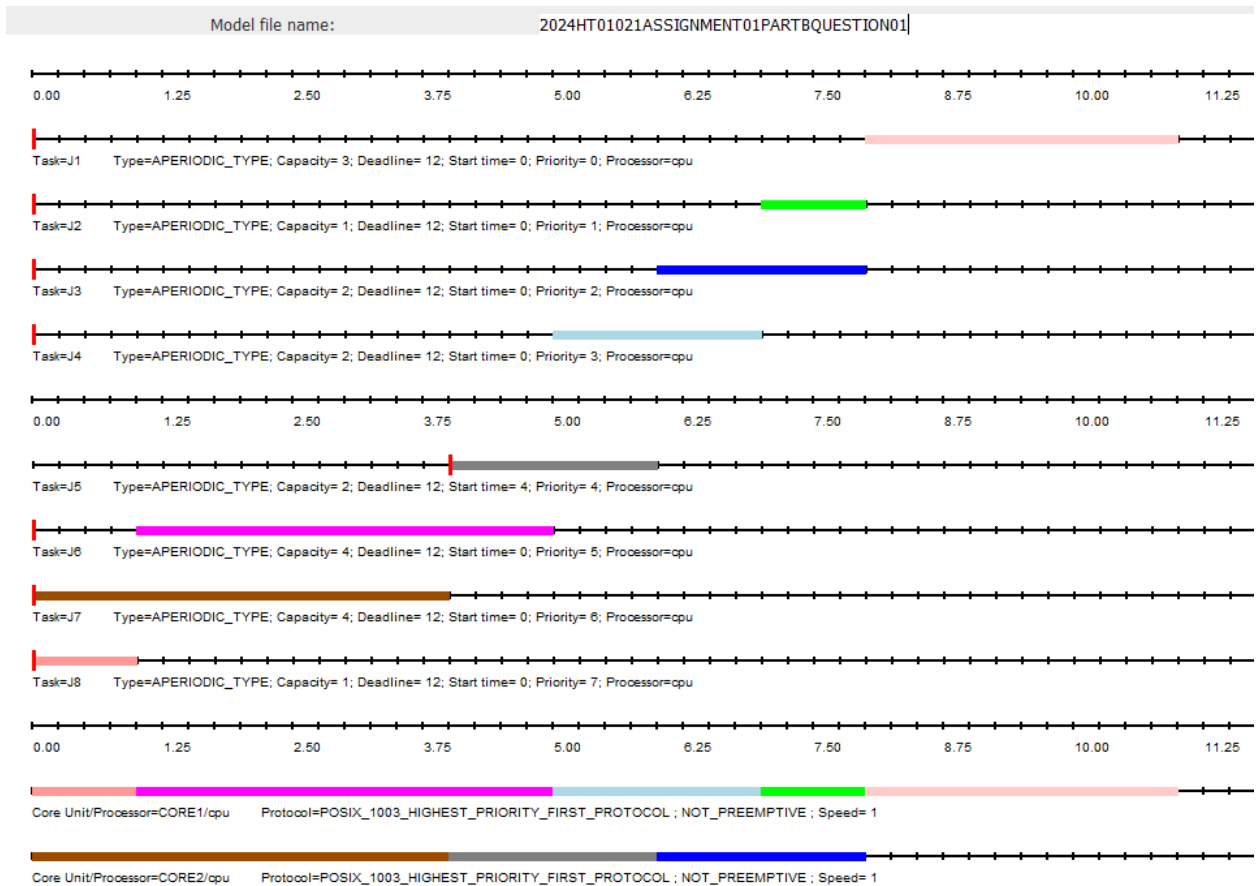
| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start time | Priority | Blocking Time |
|------|-----------|-----------|---------------|------|----------|----------|------------|----------|---------------|
| J1   | Aperiodic | cpu       | EA            |      | 3        | 12       | 0          | 0        | 0             |
| J2   | Aperiodic | cpu       | EA            |      | 1        | 12       | 0          | 1        | 0             |
| J3   | Aperiodic | cpu       | EA            |      | 2        | 12       | 0          | 2        | 0             |
| J4   | Aperiodic | cpu       | EA            |      | 2        | 12       | 0          | 3        | 0             |
| J5   | Aperiodic | cpu       | EA            |      | 2        | 12       | 4          | 4        | 0             |
| J6   | Aperiodic | cpu       | EA            |      | 4        | 12       | 0          | 5        | 0             |
| J7   | Aperiodic | cpu       | EA            |      | 4        | 12       | 0          | 6        | 0             |
| J8   | Aperiodic | cpu       | EA            |      | 1        | 12       | 0          | 7        | 0             |

### Step 5: Running the Scheduling Simulation

1. **Scheduling Setup:** Before running the simulation, we defined the time interval for scheduling (in this case, from 0 to 12).

The screenshot shows a window titled "Scheduling Simulation". Inside the window, there are three input fields with labels and values: "Schedule From 0 To : 12", "Draw From : 0", and "Draw Upto : 12". At the bottom of the window, there are two buttons labeled "Ok" and "Cancel".

2. **Executing the Simulation:** The Cheddar simulation tool was used to analyze task scheduling, showing the tasks distributed across the two cores (Core1 and Core2). The results of the simulation were displayed in the form of task Gantt charts for each task.



### 3. Results and Analysis:

- Number of context switches: 16
- Number of preemptions: 6
- Worst-case response times for each task were recorded. These results showed that no deadlines were missed, indicating that the task set is schedulable.

#### Scheduling simulation, Processor cpu :

- Number of context switches : 16
- Number of preemptions : 6
- Task response time computed from simulation :
  - J1 => 11/worst
  - J2 => 8/worst
  - J3 => 8/worst
  - J4 => 7/worst
  - J5 => 2/worst
  - J6 => 5/worst
  - J7 => 4/worst
  - J8 => 1/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

### Step 6: Analyze and Interpret the Results

1. **Gantt Chart Review:** After running the simulation, review the Gantt chart to ensure the tasks are scheduled without missing deadlines. Tasks should be divided appropriately between **CORE1** and **CORE2**.
2. **Response Time Analysis:** Analyze the worst-case response times of each task:
  - J1: 11ms
  - J2: 8ms
  - J3: 8ms
  - J4: 7ms
  - etc.

No deadlines were missed in this simulation.

3. **Conclusion:** Conclude that the system is schedulable within the given deadline using the **FIFO policy** and non-preemptive protocol. Suggest improvements, such as exploring different scheduling policies or priority adjustments.

**QUESTION 2:** Show that the following tasks table is schedulable using EDF on a mono-core processor, if pre-emption is allowed but is not schedulable if pre-emption is not allowed. All tasks are aperiodic.

| J1       | J2       | J3       |
|----------|----------|----------|
| $r = 0$  | $r = 2$  | $r = 4$  |
| $e = 3$  | $e = 6$  | $e = 4$  |
| $d = 10$ | $d = 14$ | $d = 12$ |

### Step 1: Create Core and Processor for EDF

1. **Core Setup:** Configure the core to use EDF (Earliest Deadline First) scheduling. We will set this up in both preemptive and non-preemptive modes.

#### Setup for **FIRST SIMULATION**

The screenshot shows the 'Core' configuration window. On the left, the 'CORE1' configuration is visible with the following settings:

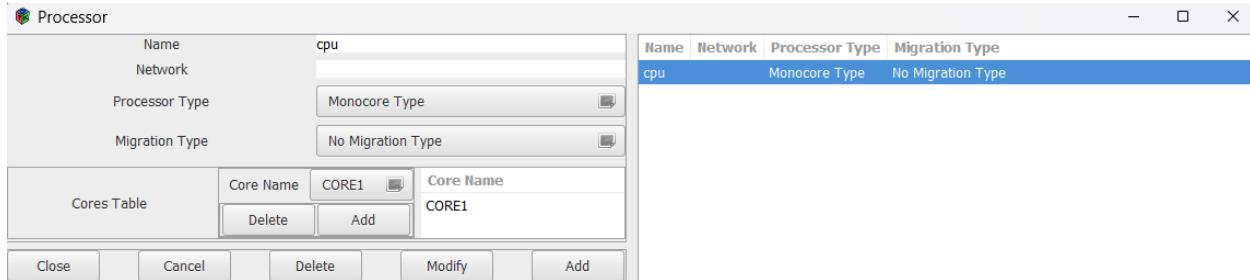
- Name: CORE1
- Scheduler type: Earliest Deadline First Protocol
- Quantum: 0
- Preemptive type: Preemptive
- Automaton name: (empty)
- Capacity: 0
- Period: 0
- Priority: 0
- User defined scheduler file name: (empty)
- Start time: 0
- Speed: 1
- L1 cache system name: (empty)

At the bottom of the left panel are buttons: Close, Cancel, Delete, Modify, and Add.

On the right, a table lists the configured cores:

| Name  | Scheduler                                  | Quantum | Preemptive     |
|-------|--------------------------------------------|---------|----------------|
| CORE1 | Earliest Deadline First Protocol           | 0       | Preemptive     |
| CORE2 | Posix 1003 Highest Priority First Protocol | 0       | Not Preemptive |

2. **Processor Setup:** Configure the processor as a **mono-core** processor, so only one core will handle the tasks.



The Processor Setup window shows configuration for a processor named 'cpu'. The Processor Type is set to 'Monocore Type' and the Migration Type is 'No Migration Type'. A table lists the cores, with 'CORE1' being the only core. Buttons for Close, Cancel, Delete, Modify, and Add are at the bottom.

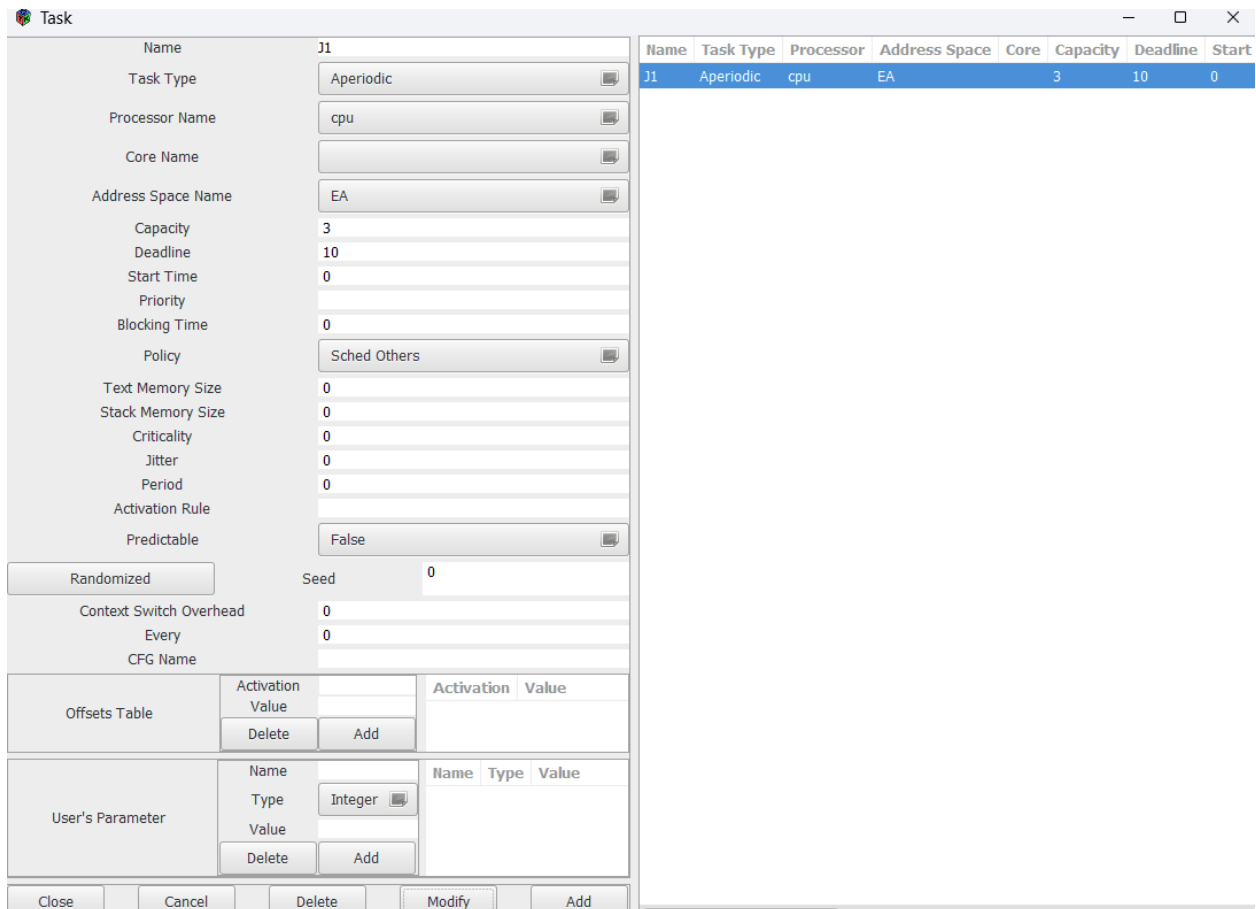
| Name | Network | Processor Type | Migration Type    |
|------|---------|----------------|-------------------|
| cpu  |         | Monocore Type  | No Migration Type |

| Core Name | Core Name |
|-----------|-----------|
| CORE1     | CORE1     |

## Step 2: Task Setup in Cheddar

We need to input the values for each task (J1, J2, J3). The values for each are as follows:

- **J1:**
  - Release Time = 0
  - Execution Time = 3
  - Deadline = 10



The Task Setup window shows configuration for a task named 'J1'. The Task Type is 'Aperiodic', Processor Name is 'cpu', Core Name is empty, Address Space Name is 'EA', Capacity is 3, Deadline is 10, Start Time is 0, Priority is 0, Blocking Time is 0, Policy is 'Sched Others', Text Memory Size is 0, Stack Memory Size is 0, Criticality is 0, Jitter is 0, Period is 0, Activation Rule is 'False', Randomized is checked, Seed is 0, Context Switch Overhead is 0, Every is 0, and CFG Name is empty. Buttons for Close, Cancel, Delete, Modify, and Add are at the bottom.

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            |      | 3        | 10       | 0     |

- **J2:**
  - Release Time = 2
  - Execution Time = 6
  - Deadline = 14

Task

Name

J2

Task Type

Aperiodic

Processor Name

cpu

Core Name

Address Space Name

EA

Capacity

6

Deadline

14

Start Time

2

Priority

0

Blocking Time

0

Policy

Sched Others

Text Memory Size

0

Stack Memory Size

0

Criticality

0

Jitter

0

Period

0

Activation Rule

Predictable

False

Randomized

Seed

0

Context Switch Overhead

0

Every

0

CFG Name

Offsets Table

Activation Value

Activation

Value

Delete

Add

User's Parameter

Name

Name

Type

Value

Type

Integer

Value

Delete

Add

Close

Cancel

Delete

Modify

Add

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            | 3    | 10       | 0        |       |
| J2   | Aperiodic | cpu       | EA            | 6    | 14       | 2        |       |
| J3   | Aperiodic | cpu       | EA            | 4    | 12       | 4        |       |

- **J3:**
  - Release Time = 4
  - Execution Time = 4
  - Deadline = 12

Task

Name

J3

Task Type

Aperiodic

Processor Name

cpu

Core Name

Address Space Name

EA

Capacity

4

Deadline

12

Start Time

4

Priority

0

Blocking Time

0

Policy

Sched Others

Text Memory Size

0

Stack Memory Size

0

Criticality

0

Jitter

0

Period

0

Activation Rule

Predictable

False

Randomized

Seed

0

Context Switch Overhead

0

Every

0

CFG Name

Offsets Table

Activation Value

Activation

Value

Delete

Add

User's Parameter

Name

Name

Type

Value

Type

Integer

Value

Delete

Add

Close

Cancel

Delete

Modify

Add

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            |      | 3        | 10       | 0     |
| J2   | Aperiodic | cpu       | EA            |      | 6        | 14       | 2     |
| J3   | Aperiodic | cpu       | EA            |      | 4        | 12       | 4     |

### Step 3: Preemptive EDF Scheduling Simulation

#### 1. Simulation Setup:

- Schedule from 0 to the latest deadline (14 in this case).
- Use preemptive scheduling (EDF with preemption).

Scheduling Simulation

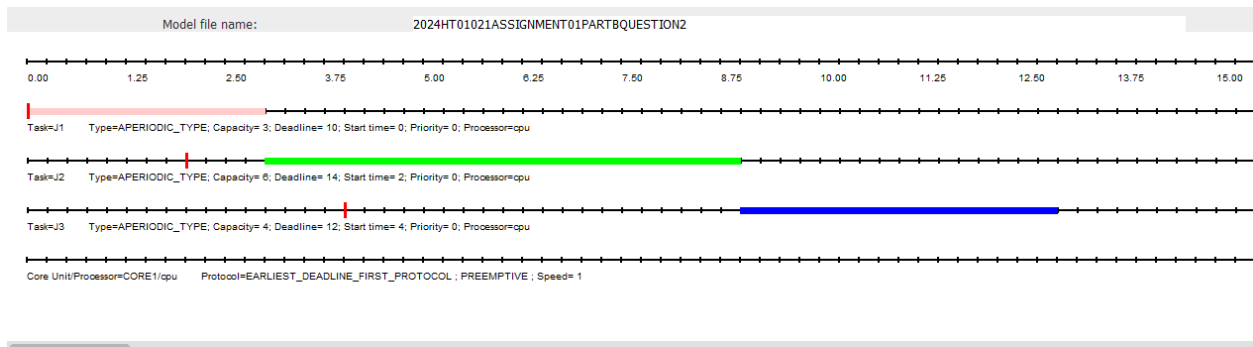
Schedule From 0 To : 14

Draw From : 0

Draw Upto : 14

Ok Cancel

- #### 2. Run Simulation:
- Once the tasks are configured, run the simulation and check if all deadlines are met.



#### Scheduling simulation, Processor cpu :

- Number of context switches : 2
- Number of preemptions : 0
- Task response time computed from simulation :
  - J1 => 3/worst
  - J2 => 7/worst
  - J3 => 9/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

All tasks completed within their deadlines, showing that the EDF scheduling worked perfectly with preemption allowed.

## Step 4: Non-Preemptive EDF Scheduling Simulation

### 1. Switch to Non-Preemptive Mode:

- Change the scheduler to non-preemptive EDF for the same core and processor setup.

#### Core Configuration:

The Core Configuration window displays the following settings for CORE1:

| Name  | Scheduler                        | Quantum | Preemptive     | Automaton name |
|-------|----------------------------------|---------|----------------|----------------|
| CORE1 | Earliest Deadline First Protocol | 0       | Not Preemptive |                |

Additional settings for CORE1:

- Quantum: 0
- Preemptive type: Not Preemptive
- Automaton name:
- Capacity: 0
- Period: 0
- Priority: 0
- User defined scheduler file name:
- Start time: 0
- Speed: 1
- L1 cache system name:

Buttons: Close, Cancel, Delete, Modify, Add

The Processor Configuration window displays the following settings for cpu:

| Name | Network | Processor Type | Migration Type    |
|------|---------|----------------|-------------------|
| cpu  |         | Monocore Type  | No Migration Type |

Core Name: CORE1

Buttons: Close, Cancel, Delete, Modify, Add

### 2. Task Configuration:

**Task**

Name: J3

Task Type: Aperiodic

Processor Name: cpu

Core Name:

Address Space Name: EA

Capacity: 4

Deadline: 12

Start Time: 4

Priority: 0

Blocking Time: 0

Policy: Sched Others

Text Memory Size: 0

Stack Memory Size: 0

Criticality: 0

Jitter: 0

Period: 0

Activation Rule:

Predictable: False

Randomized: Seed 0

Context Switch Overhead: 0

Every: 0

CFG Name:

Offsets Table:

| Activation Value | Activation | Value |
|------------------|------------|-------|
|                  |            |       |

User's Parameter:

| Name | Type    | Value |
|------|---------|-------|
|      | Integer |       |

Close Cancel Delete Modify Add

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            | 3    | 10       | 0        |       |
| J2   | Aperiodic | cpu       | EA            | 6    | 14       | 2        |       |
| J3   | Aperiodic | cpu       | EA            | 4    | 12       | 4        |       |

3. **Run Simulation:** Run the simulation again and see if any deadlines are missed. In this case, some tasks may miss their deadlines since preemption is not allowed.

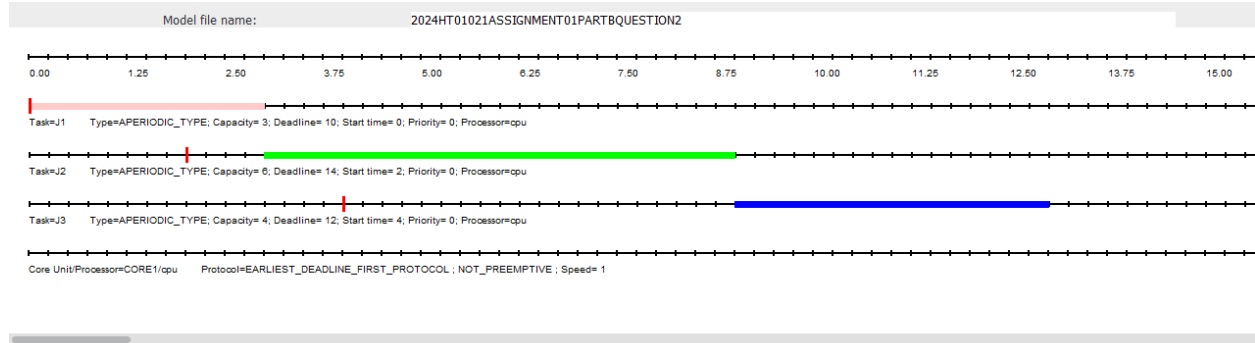
**Scheduling Simulation**

Schedule From 0 To : 14

Draw From : 0

Draw Upto : 14

Ok Cancel



#### Scheduling simulation, Processor cpu :

- Number of context switches : 2
- Number of preemptions : 0
- Task response time computed from simulation :
  - J1 => 3/worst
  - J2 => 7/worst
  - J3 => 9/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

## Step 5: Simulation Results

- **Number of Context Switches: 2**
- **Number of Preemptions: 0** (since preemption is not allowed in this case)
- **Task Response Times (Worst-Case Scenario):**
  - J1: 3 units
  - J2: 7 units
  - J3: 9 units
- **No Deadline Missed:** The task set is schedulable under the feasibility interval, meaning that the tasks completed their execution within their deadlines without missing any deadline.

This shows that the task set can be scheduled using the **Earliest Deadline First (EDF)** protocol, even without preemption.

## Conclusion:

- **Schedulable Without Preemption:** The simulation demonstrates that the tasks can meet their deadlines without requiring preemption. This is an ideal outcome since non-preemptive scheduling reduces the complexity of context switches.

**Question 3:** Show that the given task set is schedulable using **LLF (Least Laxity First)** but not using **EDF (Earliest Deadline First)** on a dual-core processor. All tasks are aperiodic.

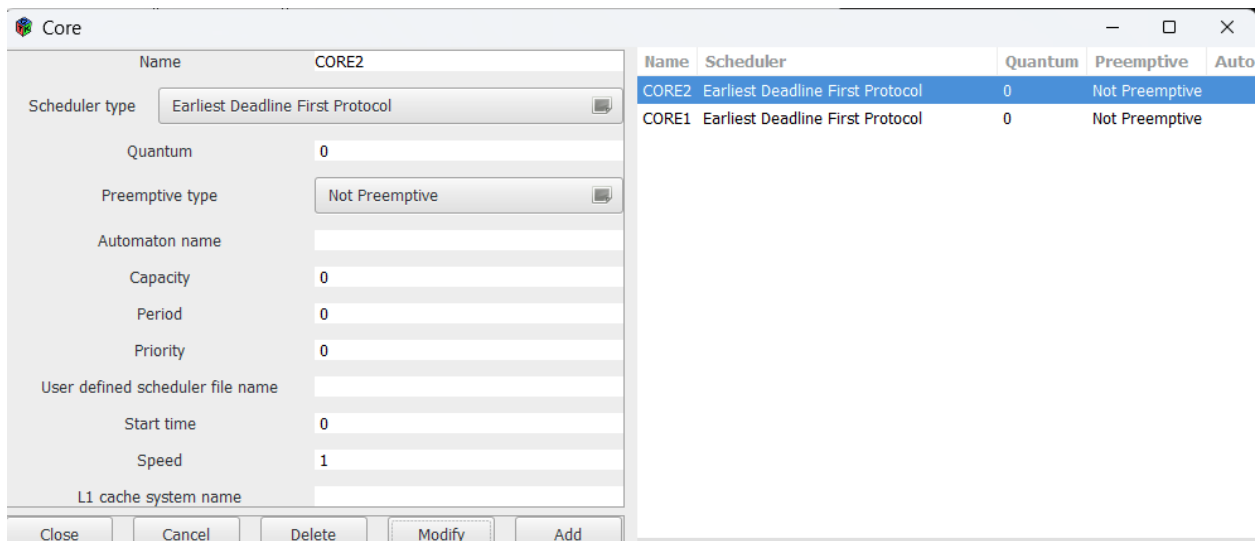
The task parameters are as follows:

|   | J1 | J2 | J3 |
|---|----|----|----|
| r | 0  | 0  | 0  |
| e | 1  | 1  | 5  |
| d | 1  | 2  | 5  |

### Step 1: Processor and Core Setup

#### 1. Core Setup:

- Both **Core 1** and **Core 2** are initially configured to use **Earliest Deadline First (EDF)** for this part of the simulation.



#### 2. Processor Setup

- Processor Type:** Identical Multicore Type

- **Migration Type: No Migration**

The Processor configuration window shows the following settings:

- Name: cpu
- Network: (empty)
- Processor Type: Identical Multicores Type
- Migration Type: No Migration Type

The Cores Table shows:

| Core Name |
|-----------|
| CORE1     |
| CORE2     |

Buttons at the bottom: Close, Cancel, Delete, Modify, Add.

## Step 2: Task Setup

Now, configure the tasks **J1**, **J2**, and **J3** in the Cheddar tool.

### Task Configuration:

#### 1. Task J1:

- **Arrival Time (r): 0**
- **Execution Time (e): 1**
- **Deadline (d): 1**

The Task configuration window shows the following settings for Task J1:

- Name: J1
- Task Type: Aperiodic
- Processor Name: cpu
- Core Name: CORE1
- Address Space Name: EA
- Capacity: 1
- Deadline: 1
- Start Time: 0
- Priority: 0
- Blocking Time: 0
- Policy: Sched Others
- Text Memory Size: 0
- Stack Memory Size: 0
- Criticality: 0
- Jitter: 0
- Period: 0
- Activation Rule: False
- Predictable: False
- Randomized: (unchecked)
- Seed: 0
- Context Switch Overhead: 0
- Every: 0
- CFG Name: (empty)

The Offsets Table and User's Parameter sections are empty.

Buttons at the bottom: Close, Cancel, Delete, Modify, Add.

The task list table on the right shows:

| Name | Task Type | Processor | Address Space | Core  | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|-------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            | CORE1 | 1        | 1        | 0     |
| J3   | Aperiodic | cpu       | EA            | CORE2 | 5        | 5        | 0     |
| J2   | Aperiodic | cpu       | EA            | CORE2 | 1        | 2        | 0     |

**2. Task J2:**

- **Arrival Time (r): 0**
- **Execution Time (e): 1**
- **Deadline (d): 2**

**Task**

Name: J2

Task Type: Aperiodic

Processor Name: cpu

Core Name: CORE2

Address Space Name: EA

Capacity: 1

Deadline: 2

Start Time: 0

Priority: 0

Blocking Time: 0

Policy: Sched Others

Text Memory Size: 0

Stack Memory Size: 0

Criticality: 0

Jitter: 0

Period: 0

Activation Rule: Predictable

Predictable: False

Randomized: Seed 0

Context Switch Overhead: 0

Every: 0

CFG Name:

Offsets Table:

| Activation Value | Activation | Value |
|------------------|------------|-------|
| Delete           | Add        |       |

User's Parameter:

| Name   | Type | Value |
|--------|------|-------|
| Delete | Add  |       |

Close Cancel Delete Modify Add

| Name | Task Type | Processor | Address Space | Core  | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|-------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            | CORE1 | 1        | 1        | 0     |
| J3   | Aperiodic | cpu       | EA            | CORE2 | 5        | 5        | 0     |
| J2   | Aperiodic | cpu       | EA            | CORE2 | 1        | 2        | 0     |

**3. Task J3:**

- **Arrival Time (r): 0**
- **Execution Time (e): 5**
- **Deadline (d): 5**

**Task**

Name: J3

Task Type: Aperiodic

Processor Name: cpu

Core Name: CORE2

Address Space Name: EA

Capacity: 5

Deadline: 5

Start Time: 0

Priority: 0

Blocking Time: 0

Policy: Sched Others

Text Memory Size: 0

Stack Memory Size: 0

Criticality: 0

Jitter: 0

Period: 0

Activation Rule: Predictable

Predictable: False

Randomized: Seed 0

Context Switch Overhead: 0

Every: 0

CFG Name:

Offsets Table:

| Activation Value | Activation | Value |
|------------------|------------|-------|
| Delete           | Add        |       |

User's Parameter:

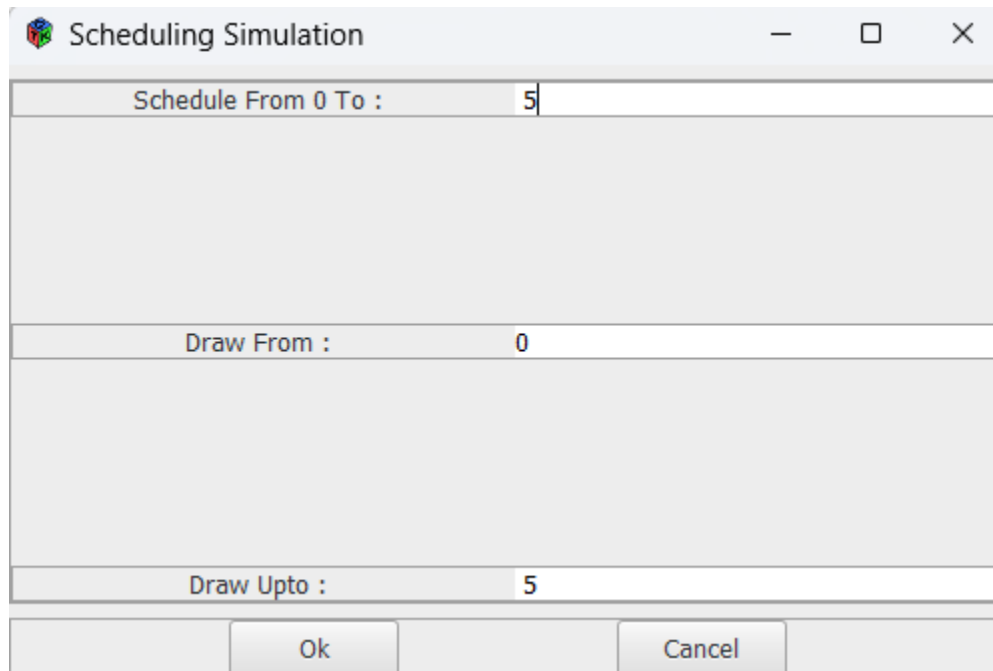
| Name   | Type | Value |
|--------|------|-------|
| Delete | Add  |       |

Close Cancel Delete Modify Add

| Name | Task Type | Processor | Address Space | Core  | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|-------|----------|----------|-------|
| J1   | Aperiodic | cpu       | EA            | CORE1 | 1        | 1        | 0     |
| J3   | Aperiodic | cpu       | EA            | CORE2 | 5        | 5        | 0     |
| J2   | Aperiodic | cpu       | EA            | CORE2 | 1        | 2        | 0     |

**Step 3: Simulation with EDF Scheduler**

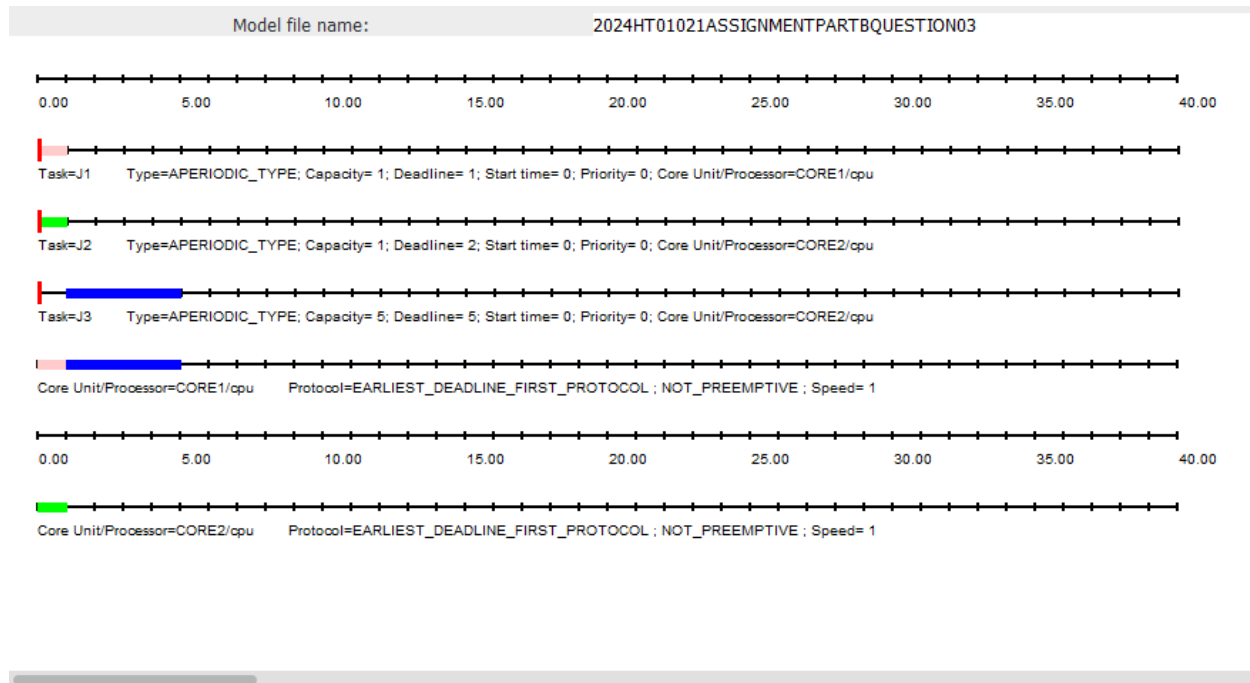
After the core and task configuration, run the scheduling simulation using **Earliest Deadline First (EDF)** scheduling to determine whether the task set is schedulable.



The screenshot shows a window titled "Scheduling Simulation" with a standard Windows-style title bar (minimize, maximize, close buttons). The window contains three input fields, each with a label and a text box:

- The first field is labeled "Schedule From 0 To :" and has the value "5" entered.
- The second field is labeled "Draw From :" and has the value "0" entered.
- The third field is labeled "Draw Upto :" and has the value "5" entered.

At the bottom of the window, there are two buttons: "Ok" and "Cancel".



#### Scheduling simulation, Processor cpu :

- Number of context switches : 2
- Number of preemptions : 0
- Task response time computed from simulation :
  - J1 => 1/worst
  - J2 => 1/worst
  - J3 => 0/worst , response time not computed since the task did not run all its capacity
- One or several tasks did not complete their execution.

#### Result:

The simulation output for EDF (Earliest Deadline First) scheduling shows that one or more tasks did not complete their execution. Specifically, Task J3 could not run to its full capacity, which is reflected in the following details:

1. **Number of Context Switches:** 2
2. **Number of Preemptions:** 0
3. **Task Response Time:**
  - o J1: 1 (worst)
  - o J2: 1 (worst)
  - o J3: 0 (worst) — Task J3 did not complete execution.

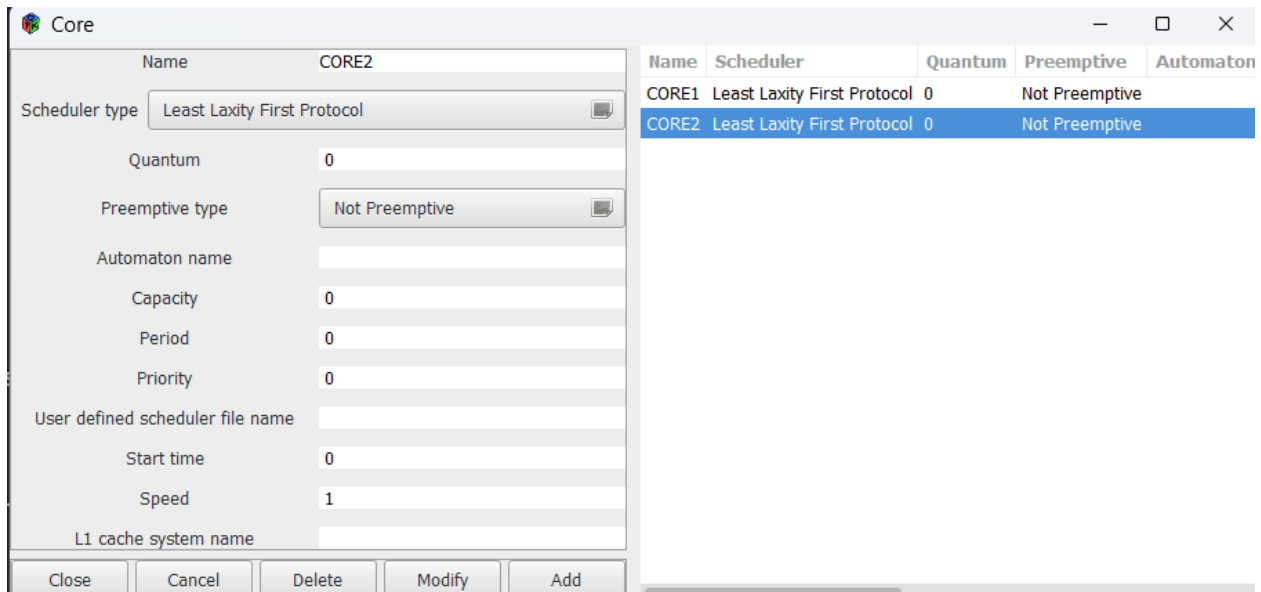
### Conclusion:

This means that under EDF scheduling with the given configuration, Task J3 could not meet its deadline, which leads to the conclusion that the system is not schedulable under EDF in this setup.

### Step 4: Change to LLF Scheduler

Next, change the scheduler to **LLF (Least Laxity First)** on both cores to verify whether the task set is schedulable under LLF.

### Core Configuration for Core 1 and Core 2.



| Name  | Scheduler                   | Quantum | Preemptive     | Automaton |
|-------|-----------------------------|---------|----------------|-----------|
| CORE1 | Least Laxity First Protocol | 0       | Not Preemptive |           |
| CORE2 | Least Laxity First Protocol | 0       | Not Preemptive |           |

### Task Setup:

Task

Name

J3

Task Type

Aperiodic

Processor Name

cpu

Core Name

CORE2

Address Space Name

EA

Capacity

5

Deadline

5

Start Time

0

Priority

0

Blocking Time

0

Policy

Sched Others

Text Memory Size

0

Stack Memory Size

0

Criticality

0

Jitter

0

Period

0

Activation Rule

Predictable

False

Randomized

Seed

0

Context Switch Overhead

0

Every

0

CFG Name

Offsets Table

Activation Value

Delete

Add

Activation

Value

User's Parameter

Name

Type

Integer

Value

Delete

Add

Name

Type

Value

Close

Cancel

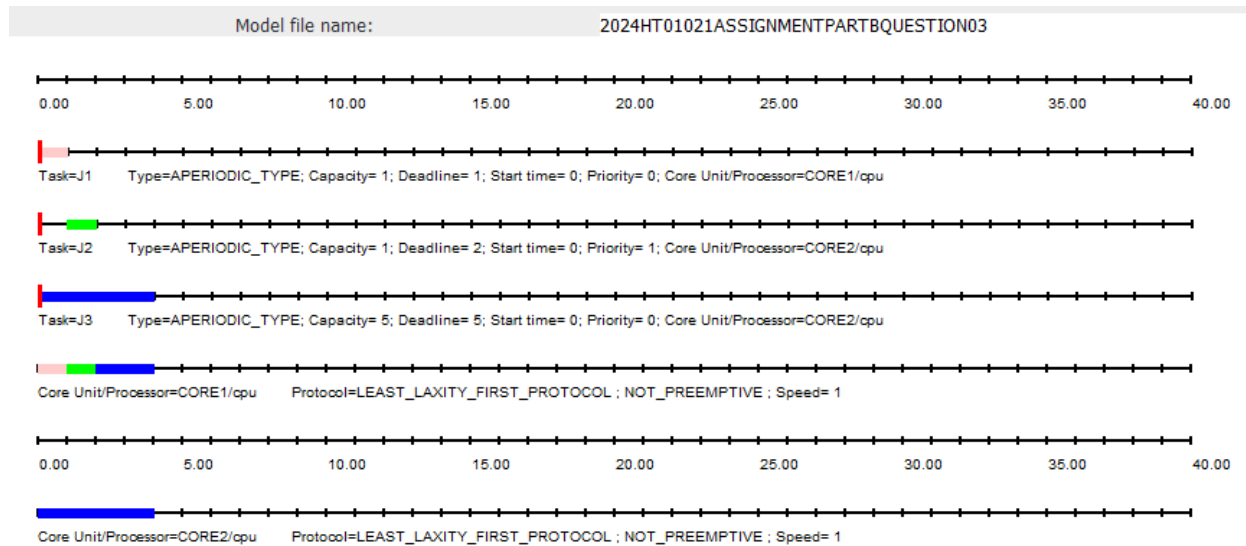
Delete

Modify

Add

| Name | Task Type | Processor | Address Space | Core  | Capacity | Deadline | Start time | Priority |   |
|------|-----------|-----------|---------------|-------|----------|----------|------------|----------|---|
| J1   | Aperiodic | cpu       | EA            | CORE1 | 1        | 1        | 0          | 0        | 0 |
| J2   | Aperiodic | cpu       | EA            | CORE2 | 1        | 2        | 0          | 1        | 0 |
| J3   | Aperiodic | cpu       | EA            | CORE2 | 5        | 5        | 0          | 0        | 0 |

### Step 5: Simulation with LLF Scheduler



- No task set ready to analyse : load a model from the 'File' menu, or edit task set from the 'Edit' menu.
- Task set ready to analyse : run analysis from the 'Tools' menu.

#### Scheduling simulation, Processor cpu :

- Number of context switches : 3
- Number of preemptions : 2
- Task response time computed from simulation :
  - J1 => 1/worst
  - J2 => 2/worst
  - J3 => 4/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

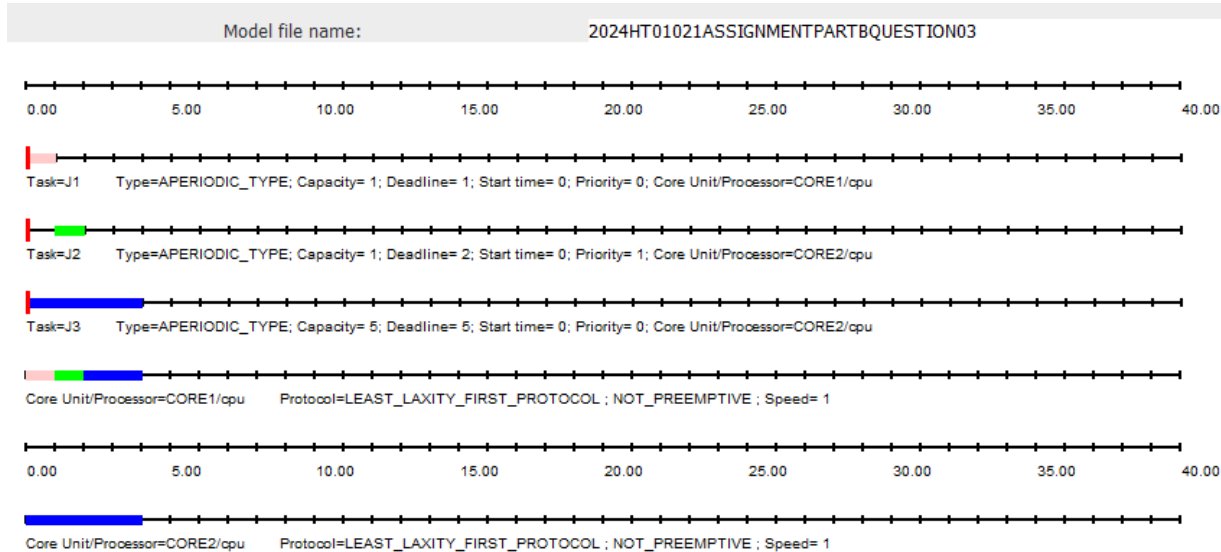
### Result:

The simulation for the tasks J1, J2, and J3 has run successfully. The Least Laxity First (LLF) protocol has been applied, and the output confirms the following:

- Number of Context Switches:** 3
- Number of Preemptions:** 2
- Task Response Times:**
  - J1: 1 (Worst-case response time)
  - J2: 2 (Worst-case response time)
  - J3: 4 (Worst-case response time)

### Conclusion:

The task set is **schedulable** on the dual-core processor under the LLF protocol, and no deadlines were missed in the computed scheduling.



### Step 6: Analysis and Conclusion

In this analysis, we demonstrated that **Earliest Deadline First (EDF)** scheduling fails to meet the deadline of Task J3 because it prioritizes J1 and J2, both of which have shorter deadlines. As a result, J3's execution is delayed beyond its deadline, making the task set unschedulable under EDF.

On the other hand, **Least Laxity First (LLF)** successfully schedules all tasks because it prioritizes tasks with the least slack, ensuring that Task J3's long execution time is accounted for without it missing its deadline.

### Conclusion:

- The task set is schedulable using **LLF** but not using **EDF** on a dual-core processor.

**QUESTION 8** For the following Task set and scheduling algorithms – submit the screen shot of the schedule generated by Cheddar – with proper interpretations and conclusions regarding the schedule generated.

| TASKS :          | A  | B  | C  | D  | E  | F  | G  | H  | I  | J   |
|------------------|----|----|----|----|----|----|----|----|----|-----|
| COMPUTATION TIME | 5  | 7  | 3  | 1  | 10 | 16 | 1  | 13 | 9  | 17  |
| TIME PERIOD      | 10 | 21 | 22 | 24 | 30 | 40 | 50 | 55 | 75 | 100 |

**1. Scheduling Algorithm Selection:**

- I used different scheduling algorithms like Rate Monotonic Scheduling (RMS) and Earliest Deadline First (EDF) to analyze the schedule of the given tasks.
- I entered the given computation times and periods for each task in Cheddar.

**2. Simulation and Analysis:**

- I input all tasks into Cheddar with their respective computation times and periods.
- I simulated using both RMS and EDF to observe the generated schedule.
- I took screenshots of the schedules generated by Cheddar for both algorithms.

**Rate Monotonic Protocol Analysis:** I started analyzing the given task set using the Rate Monotonic Scheduling (RMS) protocol. RMS is a static priority scheduling algorithm where tasks with shorter periods are assigned higher priorities. The initial observations from Cheddar indicate that several tasks did not complete their execution within their deadlines due to the high utilization factor of the system. Specifically, tasks E, F, G, H, I, and J did not run all their required capacity, which means they missed their deadlines. This suggests that the system utilization is too high, and the given set of tasks cannot be feasibly scheduled under RMS.

The total utilization of the task set was calculated, and it exceeded the theoretical bound for RMS feasibility, which is given by:

- **Feasibility Condition for RMS:**

Since the utilization exceeded this bound, RMS could not guarantee that all tasks would meet their deadlines. This was further validated in the Cheddar simulation, where several tasks failed to execute their full capacity.

**Simulation Steps for Initial Setup:**

- **Step 1:** Open the Cheddar tool.
- **Step 2:** Create a new project and enter the given task set details, including computation times and periods.
- **Step 3:** Add a processor core for scheduling and set the properties

**Core**

Name: CORE1

Scheduler type: Rate Monotonic Protocol

Quantum: 0

Preemptive type: Preemptive

Automaton name:

Capacity: 0

Period: 0

Priority: 0

User defined scheduler file name:

Start time: 0

Speed: 1

L1 cache system name:

Buttons: Close, Cancel, Delete, Modify, Add

| Name  | Scheduler               | Quantum | Preemptive | Automaton |
|-------|-------------------------|---------|------------|-----------|
| CORE1 | Rate Monotonic Protocol | 0       | Preemptive |           |

- **Step 4:** Create a processor and assign tasks to the created core.

**Processor**

Name: cpu

Network:

Processor Type: Monocore Type

Migration Type: No Migration Type

Cores Table:

| Core Name | Core Name |
|-----------|-----------|
| CORE1     | CORE1     |

Buttons: Close, Cancel, Delete, Modify, Add

| Name | Network | Processor Type | Migration Type    |
|------|---------|----------------|-------------------|
| cpu  |         | Monocore Type  | No Migration Type |

- **Step 5:** Select the scheduling algorithm as Rate Monotonic (RMS).
- **Step 6:** Create Task

**Task**

Name: J

Task Type: Periodic

Processor Name: cpu

Core Name:

Address Space Name: EA

Capacity: 17

Deadline: 100

Start Time: 0

Priority: 10

Blocking Time: 0

Policy: Sched Fifo

Text Memory Size: 0

Stack Memory Size: 0

Criticality: 0

Jitter: 0

Period: 100

Activation Rule:

Predictable: False

Randomized: Seed 0

Context Switch Overhead: 0

Every: 0

CFG Name:

| Offsets Table | Activation Value | Activation | Value |
|---------------|------------------|------------|-------|
|               | Delete           | Add        |       |

| User's Parameter | Name | Type    | Value |
|------------------|------|---------|-------|
|                  |      | Integer |       |
|                  |      |         |       |
|                  |      |         |       |

Close Cancel Delete Modify Add

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start |
|------|-----------|-----------|---------------|------|----------|----------|-------|
| A    | Periodic  | cpu       | EA            | 5    | 10       | 0        | 0     |
| B    | Periodic  | cpu       | EA            | 7    | 21       | 0        | 0     |
| C    | Periodic  | cpu       | EA            | 3    | 22       | 0        | 0     |
| D    | Periodic  | cpu       | EA            | 1    | 24       | 0        | 0     |
| E    | Periodic  | cpu       | EA            | 10   | 30       | 0        | 0     |
| F    | Periodic  | cpu       | EA            | 16   | 40       | 0        | 0     |
| G    | Periodic  | cpu       | EA            | 1    | 50       | 0        | 0     |
| H    | Periodic  | cpu       | EA            | 13   | 55       | 0        | 0     |
| I    | Periodic  | cpu       | EA            | 9    | 75       | 0        | 0     |
| J    | Periodic  | cpu       | EA            | 17   | 100      | 0        | 0     |

- **Step 7:** Run the simulation and observe the generated schedule.

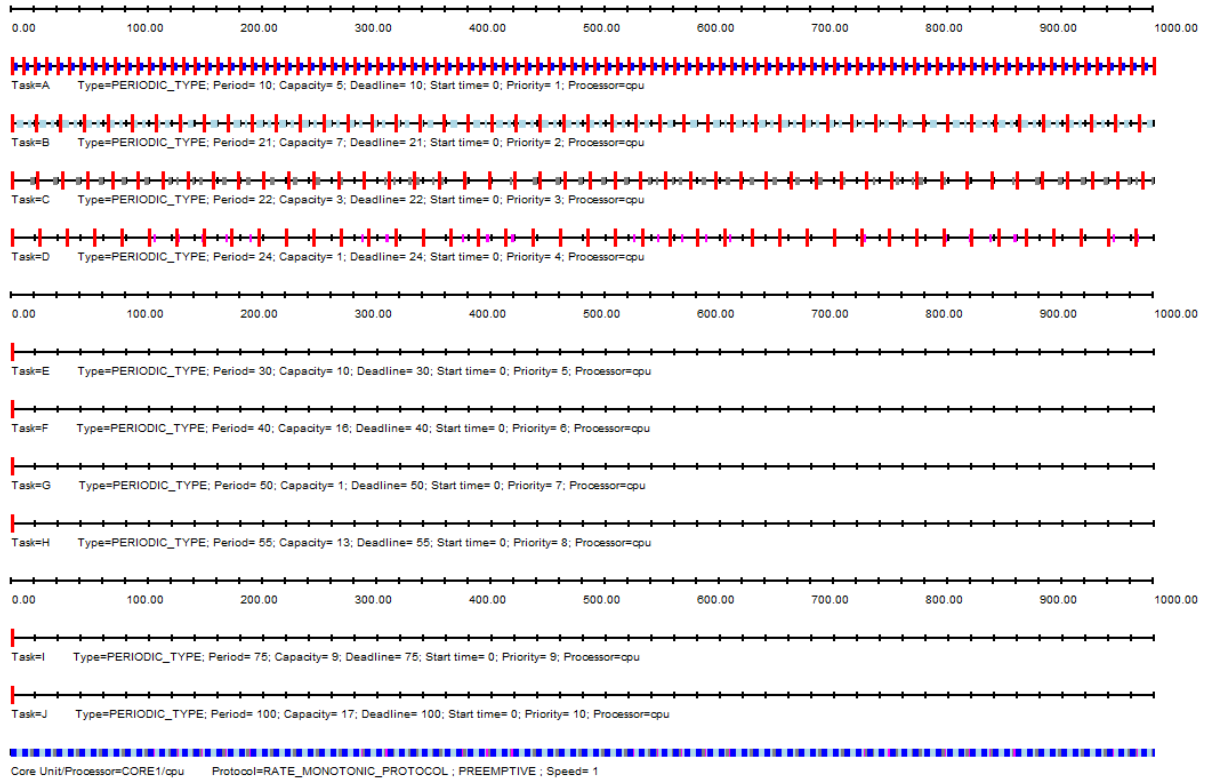
Cheddar: a free real time scheduling simulator

File Edit Tools Help



Model file name:

2024HT01021ASSIGNMENT01EXP02

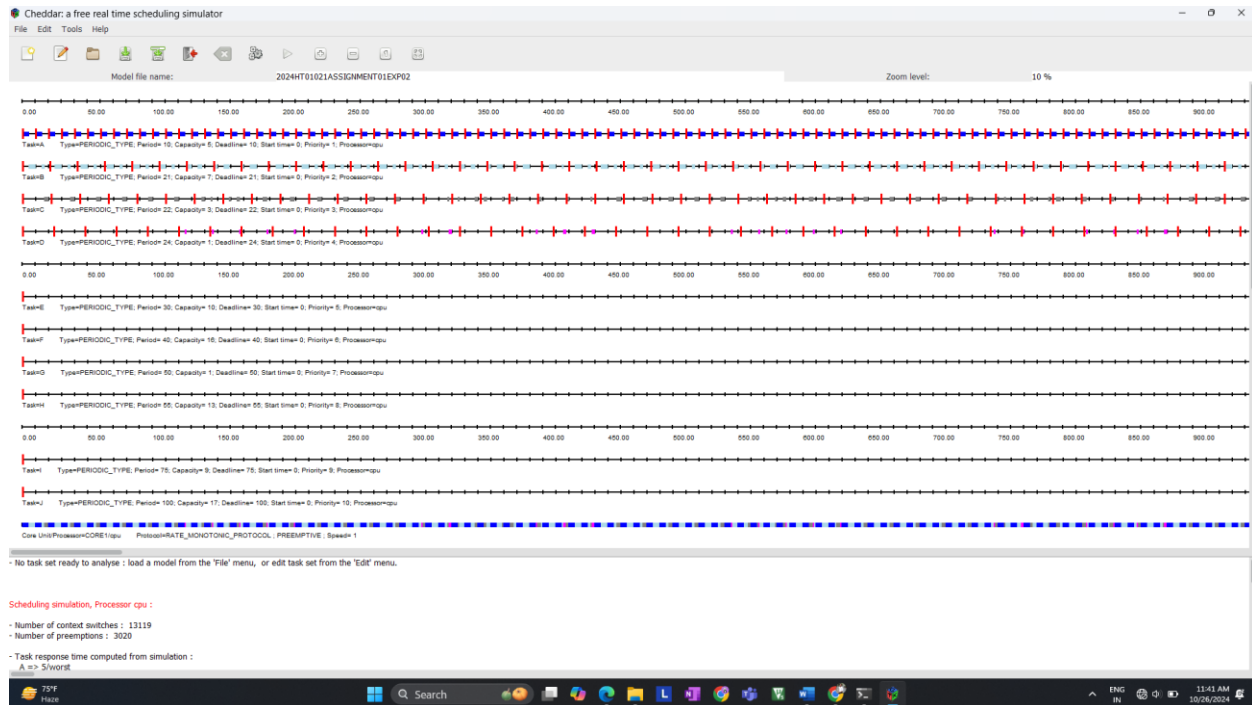


- No task set ready to analyse : load a model from the 'File' menu, or edit task set from the 'Edit' menu.

Scheduling simulation, Processor cpu :

- Number of context switches : 13119  
- Number of preemptions : 3020

- Task response time computed from simulation :  
A => 5/worst



No task set ready to analyse : load a model from the 'File' menu, or edit task set from the 'Edit' menu.

Scheduling simulation, Processor cpu :

- Number of context switches : 13119
- Number of preemptions : 3020

Task response time computed from simulation :

- A => 5/worst
- B => 17/worst
- C => 20/worst
- D => 12676/worst, missed its deadline (absolute deadline = 24 ; completion time = 126), missed its deadline (absolute deadline = 48 ; completion time = 147), missed its deadline (absolute deadline = 72 ; completion time = 168), missed its deadline (absolute deadline = 96 ; completion time = 189), missed its deadline (
- E => 0/worst, response time not computed since the task did not run all its capacity
- F => 0/worst, response time not computed since the task did not run all its capacity
- G => 0/worst, response time not computed since the task did not run all its capacity
- H => 0/worst, response time not computed since the task did not run all its capacity
- I => 0/worst, response time not computed since the task did not run all its capacity
- J => 0/worst, response time not computed since the task did not run all its capacity

One or several tasks did not complete their execution.

## Interpretations and Conclusions:

- RMS Analysis:** Under Rate Monotonic Scheduling, tasks with shorter periods are given higher priorities. In this case, due to the high utilization of the processor, several tasks could not meet their deadlines. This is indicative of a task set that exceeds the capacity that RMS can feasibly handle.

**Next Steps:** I will continue with the analysis by using the Earliest Deadline First (EDF) scheduling algorithm to determine if it can handle the given task set more effectively.

**Earliest Deadline First (EDF) Analysis:** Next, I analyzed the given task set using the Earliest Deadline First (EDF) protocol. EDF is a dynamic priority scheduling algorithm where tasks are assigned priorities based on their absolute deadlines — tasks with the earliest deadlines are assigned the highest priority.

## Simulation Steps for EDF Setup:

- **Step 1:** Open the Cheddar tool and create a new project.
- **Step 2:** Define a new core named "CORE1" and select "Earliest Deadline First Protocol" as the scheduler type.

| Name  | Scheduler                        | Quantum | Preemptive | Automa |
|-------|----------------------------------|---------|------------|--------|
| CORE1 | Earliest Deadline First Protocol | 0       | Preemptive |        |

- **Step 3:** Create a processor named "cpu" and assign "CORE1" to it.

| Name | Network | Processor Type | Migration Type    |
|------|---------|----------------|-------------------|
| cpu  |         | Monocore Type  | No Migration Type |

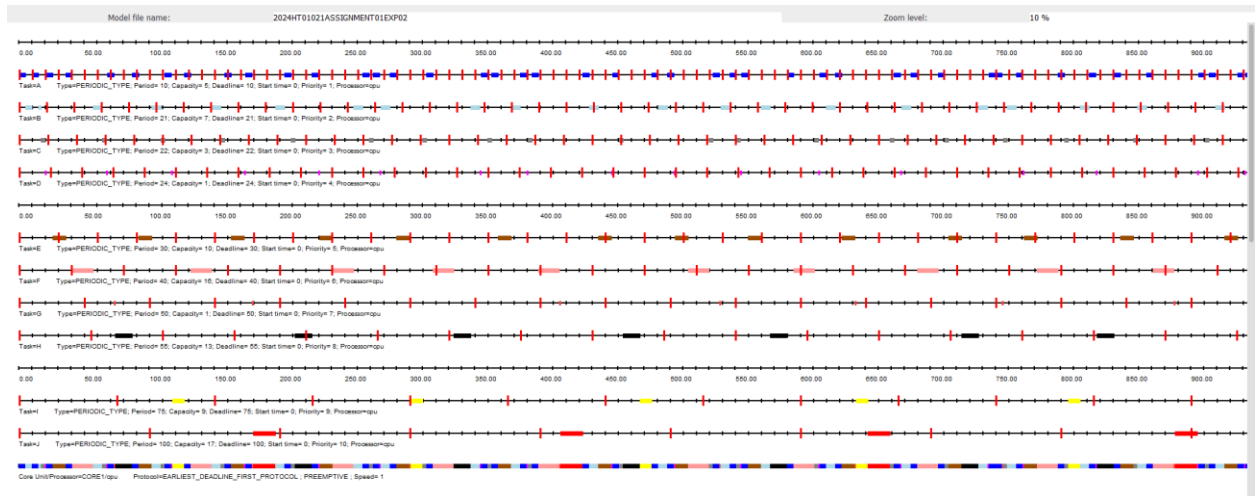
- **Step 4:** Create an address space named "EA" and associate it with the processor "cpu".

| Name | Processor Name | Text Size | Stack Size | Data Size | Heap Size | Scheduler              | Quantum |
|------|----------------|-----------|------------|-----------|-----------|------------------------|---------|
| EA   | cpu            | 0         | 0          | 0         | 0         | No Scheduling Protocol | 0       |

- **Step 5:** Enter the task details (tasks A to J) into Cheddar, associating them with the address space "EA" and specifying their properties such as capacity, period, and priority.

| Name | Task Type | Processor | Address Space | Core | Capacity | Deadline | Start time | Priority | Blocking Time | Policy     | Text |
|------|-----------|-----------|---------------|------|----------|----------|------------|----------|---------------|------------|------|
| A    | Periodic  | cpu       | EA            |      | 5        | 10       | 0          | 1        | 0             | Sched Fifo | 0    |
| B    | Periodic  | cpu       | EA            |      | 7        | 21       | 0          | 2        | 0             | Sched Fifo | 0    |
| C    | Periodic  | cpu       | EA            |      | 3        | 22       | 0          | 3        | 0             | Sched Fifo | 0    |
| D    | Periodic  | cpu       | EA            |      | 1        | 24       | 0          | 4        | 0             | Sched Fifo | 0    |
| E    | Periodic  | cpu       | EA            |      | 10       | 30       | 0          | 5        | 0             | Sched Fifo | 0    |
| F    | Periodic  | cpu       | EA            |      | 16       | 40       | 0          | 6        | 0             | Sched Fifo | 0    |
| G    | Periodic  | cpu       | EA            |      | 1        | 50       | 0          | 7        | 0             | Sched Fifo | 0    |
| H    | Periodic  | cpu       | EA            |      | 13       | 55       | 0          | 8        | 0             | Sched Fifo | 0    |
| I    | Periodic  | cpu       | EA            |      | 9        | 75       | 0          | 9        | 0             | Sched Fifo | 0    |
| J    | Periodic  | cpu       | EA            |      | 17       | 100      | 0          | 10       | 0             | Sched Fifo | 0    |

**Simulation Results for EDF:** Upon running the simulation, the generated schedule for EDF showed that all tasks still missed their deadlines, similar to the RMS analysis. Here are the key details from the EDF simulation:



Task: random time computed from simulation:

A = 26041/vorst, missed its deadline (absolute deadline = 40; completion time = 41), missed its deadline (absolute deadline = 50; completion time = 73), missed its deadline (absolute deadline = 60; completion time = 82), missed its deadline (absolute deadline = 70; completion time = 117), missed its deadline (abs

B = 26031/vorst, missed its deadline (absolute deadline = 48; completion time = 64), missed its deadline (absolute deadline = 62; completion time = 109), missed its deadline (absolute deadline = 84; completion time = 155), missed its deadline (absolute deadline = 105; completion time = 204), missed its deadline (

C = 26021/vorst, missed its deadline (absolute deadline = 48; completion time = 66), missed its deadline (absolute deadline = 66; completion time = 112), missed its deadline (absolute deadline = 88; completion time = 158), missed its deadline (absolute deadline = 110; completion time = 213), missed its deadli

D = 26011/vorst, missed its deadline (absolute deadline = 48; completion time = 68), missed its deadline (absolute deadline = 72; completion time = 118), missed its deadline (absolute deadline = 96; completion time = 174), missed its deadline (absolute deadline = 120; completion time = 231), missed its deadli

E = 26001/vorst, missed its deadline (absolute deadline = 48; completion time = 70), missed its deadline (absolute deadline = 72; completion time = 120), missed its deadline (absolute deadline = 96; completion time = 180), missed its deadline (absolute deadline = 120; completion time = 234), missed its deadli

F = 26042/vorst, missed its deadline (absolute deadline = 48; completion time = 57), missed its deadline (absolute deadline = 80; completion time = 148), missed its deadline (absolute deadline = 120; completion time = 257), missed its deadline (absolute deadline = 160; completion time = 334), missed its deadli

G = 26020/vorst, missed its deadline (absolute deadline = 50; completion time = 74), missed its deadline (absolute deadline = 100; completion time = 180), missed its deadline (absolute deadline = 150; completion time = 301), missed its deadline (absolute deadline = 200; completion time = 416), missed its deadli

H = 26010/vorst, missed its deadline (absolute deadline = 50; completion time = 76), missed its deadline (absolute deadline = 100; completion time = 182), missed its deadline (absolute deadline = 150; completion time = 303), missed its deadline (absolute deadline = 200; completion time = 418), missed its deadli

I = 26079/vorst, missed its deadline (absolute deadline = 75; completion time = 127), missed its deadline (absolute deadline = 150; completion time = 310), missed its deadline (absolute deadline = 225; completion time = 486), missed its deadline (absolute deadline = 300; completion time = 652), missed its deadli

J = 26039/vorst, missed its deadline (absolute deadline = 100; completion time = 197), missed its deadline (absolute deadline = 200; completion time = 433), missed its deadline (absolute deadline = 300; completion time = 669), missed its deadline (absolute deadline = 400; completion time = 905), missed its deadli

- KARVE ASHUTOSH RAJENDRA 44

- **Preemptions:** Only 1 preemption occurred during the scheduling.

**Task Response Times and Deadlines:** The EDF simulation results indicated that all tasks missed their deadlines multiple times throughout the simulation period. The following table summarizes the task response times and the missed deadlines:

**Generated Schedule Visualization:** The graphical representation of the schedule shows how tasks were executed over time. It is evident that tasks frequently missed their deadlines, as indicated by the red segments.

#### **Interpretations and Conclusions for EDF:**

- **EDF Analysis:** Under EDF scheduling, tasks are assigned dynamic priorities based on their deadlines. Despite EDF being more efficient than RMS in some cases, the high processor utilization and the complexity of the task set led to multiple missed deadlines. The system's total utilization exceeds what can be feasibly managed, even with dynamic priority adjustments. This outcome demonstrates that neither RMS nor EDF can feasibly schedule the given task set without missing deadlines.

- END -  
-